

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze	5
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze	5
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply	5
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze	5
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply	5
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze	5
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply	5

8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply	5
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze	5
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze	5

Code of Conduct:

I G Vyshnavi (192525260) __ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: vyshnavi

RUBRICS FOR CALCULATION:

Constraint-Based Problem Solving Assessment Rubric					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of	15%	Effectively	Applies	Limited	Unable to

Concepts		applies appropriate concepts, methods, or tools	concepts with minor errors	application; requires guidance	apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Q1. Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

Consider the query:

```
SELECT S.Student_ID, S.Name
FROM Student S
JOIN Enrollment E ON S.Student_ID = E.Student_ID
WHERE E.Course_ID IN
  (SELECT C.Course_ID
   FROM Course C
   WHERE C.Department = 'CSE');
```

Initial Query Structure

The query contains:

- A join between Student and Enrollment.
- A sub-query on Course.
- A selection condition Department = 'CSE'.
- A final selection based on the course IDs returned by the sub-query.

Heuristic Optimization Steps

Step 1: Push selections down

Apply the selection Department = 'CSE' directly to the Course relation.

$\sigma_{\text{Department}='CSE'}(\text{Course})$

This reduces the number of Course tuples before further processing.

Step 2: Replace the sub-query with an equivalent join

The optimized query can be represented using joins between the filtered Course relation and Enrollment.

$\text{Student} \bowtie \text{Enrollment} \bowtie \sigma_{\text{Department}='CSE'}(\text{Course})$

Step 3: Push projections down

Only required attributes are retained:

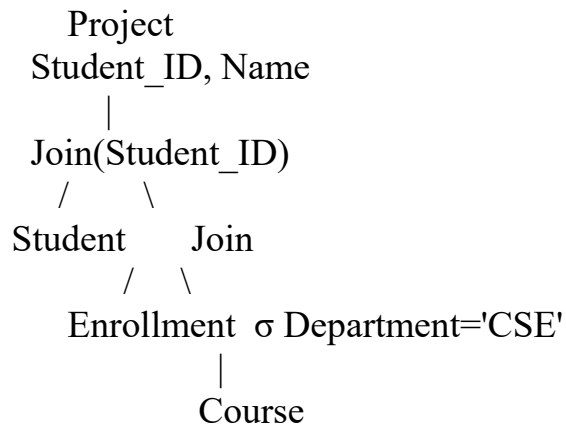
$\text{Student} \rightarrow \text{Student_ID}, \text{Name}$

Enrollment $\rightarrow$ Student_ID, Course_ID
Course $\rightarrow$ Course_ID

Step 4: Perform smaller operations first

The filtered Course relation is joined with Enrollment first because it contains fewer tuples.

Optimized Execution Plan



Conclusion

The optimized plan reduces intermediate results by pushing selections and projections down and performing the smaller join first. Therefore, disk I/O and processing cost are reduced.

Q2. Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.

Assume:

- Relation R = 10,000 tuples
- Relation S = 2,000 tuples
- Page size = 100 tuples
- Therefore:
 - R = 100 pages
 - S = 20 pages

1. Nested-Loop Join

If S is used as the outer relation:

Cost $\approx B(S) + B(S) \times B(R)$
= $20 + (20 \times 100)$
= 2020 page accesses

2. Sort-Merge Join

Both relations must be sorted before merging.

For these relatively moderate relation sizes, sorting introduces additional disk I/O. Therefore, its cost is higher than a simple hash join when the join condition is an equality condition.

3. Hash Join

For an equality join, a hash join can partition the relations using the join attribute and compare matching partitions.

Since S is only 20 pages and R is 100 pages, hash join requires significantly fewer accesses than the basic nested-loop approach.

Comparison

Join Method	Main Cost	Suitability
Nested-Loop	High repeated page access	Suitable for very small outer relation
Sort-Merge	Sorting + merging	Useful for sorted data and range operations
Hash Join	Partitioning + matching	Very efficient for equality joins

Decision

For the given relations and an **equality join**, I would choose **Hash Join** because it avoids repeatedly scanning the larger relation and does not require sorting.

The main constraints are relation size, page count, available memory, and type of join condition. Since the smaller relation has only 20 pages, hash-based processing is efficient.

Q3. Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.

JDBC can be used to connect a Java application to MySQL. A PreparedStatement is used for parameterized queries and helps prevent SQL injection.

```
import java.sql.*;
```

```
public class JDBCExample {
```

```

public static void main(String[] args) {

    String url = "jdbc:mysql://localhost:3306/college";
    String user = "root";
    String password = "root";

    String sql = "SELECT * FROM Student WHERE Department = ?";

    try {
        Connection con = DriverManager.getConnection(
            url, user, password);

        PreparedStatement ps = con.prepareStatement(sql);

        ps.setString(1, "CSE");

        ResultSet rs = ps.executeQuery();

        while (rs.next()) {
            System.out.println(
                rs.getInt("Student_ID") + " " +
                rs.getString("Name"));
        }

        rs.close();
        ps.close();
        con.close();

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

Explanation

1. DriverManager.getConnection() establishes the database connection.
2. PreparedStatement creates a parameterized SQL query.
3. setString() supplies the required parameter.
4. executeQuery() executes the SELECT statement.
5. ResultSet stores the returned records.
6. Finally, resources are closed.

This approach is feasible because it provides secure and reusable database connectivity.

Q4. Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.

Consider the following schedule:

$S = R_1(A) \ W_1(A)$

$R_2(A) \ W_2(B)$

$R_3(B) \ W_3(C)$

$R_4(C) \ W_4(A)$

Step 1: Identify conflicts

For the same data item:

- T_1 writes A before T_2 reads A $\rightarrow T_1 \rightarrow T_2$
- T_2 writes B before T_3 reads B $\rightarrow T_2 \rightarrow T_3$
- T_3 writes C before T_4 reads C $\rightarrow T_3 \rightarrow T_4$
- T_4 writes A after T_1 writes A $\rightarrow T_1 \rightarrow T_4$

Step 2: Construct the precedence graph

$T_1 \longrightarrow T_2 \longrightarrow T_3 \longrightarrow T_4$
 $\backslash \hspace{10em} \longrightarrow$

Edges are:

$T_1 \rightarrow T_2$

$T_2 \rightarrow T_3$

$T_3 \rightarrow T_4$

$T_1 \rightarrow T_4$

Step 3: Check for cycle

There is **no cycle** in the graph.

Result

Since the precedence graph is acyclic, the schedule is **conflict-serializable**.

One equivalent serial order is:

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4$

Therefore, the schedule satisfies the serializability constraint.

Q5. Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.

Consider:

T1: Read(A), Write(A), Read(B), Write(B)

T2: Read(B), Write(B), Read(A), Write(A)

Basic 2PL

Basic 2PL has two phases:

1. **Growing Phase:** Transactions can acquire locks but cannot release them.
2. **Shrinking Phase:** Transactions can release locks but cannot acquire new locks.

Possible execution

T1: X-lock(A)

T1: Read(A)

T1: Write(A)

T2: X-lock(B)

T2: Read(B)

T2: Write(B)

T1: Request X-lock(B) → WAIT

T2: Request X-lock(A) → WAIT

Now:

T1 waits for B held by T2

T2 waits for A held by T1

This creates a deadlock.

Wait-For Graph



There is a cycle:

$T1 \rightarrow T2 \rightarrow T1$

Therefore, a deadlock exists.

Resolution

One transaction can be rolled back.

For example:

Abort T2

Release B

T1 obtains X-lock(B)

T1 completes

T1 commits

T2 restarts

Conclusion

Basic 2PL ensures conflict serializability, but it does not completely prevent deadlocks. Deadlock detection and transaction rollback can be used to resolve them.

Q6. Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes.

Assume:

T1 timestamp = 10 → Older

T2 timestamp = 20 → Younger

Suppose T1 requests a lock held by T2.

Wait-Die

In Wait-Die:

- Older transaction can wait for younger transaction.
- Younger transaction requesting a lock held by older transaction is rolled back.

Therefore:

T1 (older) requests lock held by T2

→ T1 waits

If T2 requests a lock held by T1:

T2 (younger) requests lock held by T1
→ T2 is rolled back

Wound-Wait

In Wound-Wait:

- Older transaction can force a younger transaction to roll back.
- Younger transaction waits for an older transaction.

Therefore:

T1 (older) requests lock held by T2
→ T2 is rolled back

If T2 requests a lock held by T1:

T2 (younger) requests lock held by T1
→ T2 waits

Comparison

Situation	Wait-Die	Wound-Wait
Older requests younger's lock	Older waits	Younger is rolled back
Younger requests older's lock	Younger rolls back	Younger waits

Both methods prevent circular waiting and therefore help prevent deadlocks.

Q7. Given transaction logs with checkpoints, apply Immediate Update recovery after a system failure.

Consider the log:

<START T1>
<T1, A, 100, 150>

<START T2>
<T2, B, 200, 250>

<COMMIT T1>

<CHECKPOINT>

<START T3>
<T3, C, 300, 350>

<COMMIT T3>

SYSTEM FAILURE

Step 1: Identify committed transactions

Before failure:

T1 → COMMITTED
T2 → UNCOMMITTED
T3 → COMMITTED

Step 2: Immediate Update recovery

In Immediate Update, changes may reach the database before commit. Therefore:

- Committed transactions require **REDO**.
- Uncommitted transactions require **UNDO**.

Step 3: REDO committed transactions

REDO T1:
A = 150

REDO T3:
C = 350

Step 4: UNDO uncommitted transactions

UNDO T2:
B = 200

Final database state

A = 150
B = 200
C = 350

Conclusion

The checkpoint reduces the amount of log that must be examined. Immediate Update requires both UNDO and REDO depending on transaction status.

Q8. Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file.

Consider:

<START T1>
<T1, A, 100, 150>

<START T2>
<T2, B, 200, 250>

<COMMIT T1>

<START T3>
<T3, C, 300, 350>

SYSTEM FAILURE

Step 1: Identify transaction status

T1 → Committed
T2 → Uncommitted
T3 → Uncommitted

Step 2: Apply Deferred Update

In Deferred Update, database changes are not written to the database until the transaction commits.

Therefore:

- Committed transactions → **REDO**
- Uncommitted transactions → ignored
- **UNDO is not required**

Step 3: REDO T1

A = 150

T2 and T3 are uncommitted, so their changes are not applied.

Final result

A = 150

B = 200

C = 300

Conclusion

Deferred Update follows the **NO-UNDO/REDO** approach. Only committed transactions are redone after a crash, making recovery simpler.

Q9. Design a concurrency control mechanism using Strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.

Consider a bank transfer:

T1: Transfer ₹100 from Account A to Account B

T2: Read balance of Account A and Account B

Strict 2PL Design

T1 first obtains exclusive locks:

T1:

X-lock(A)

X-lock(B)

Read A

A = A - 100

Write A

Read B

B = B + 100

Write B

COMMIT

Unlock(A)
Unlock(B)

T2 attempts to read the balance while T1 is updating:

T2:
S-lock(A) → WAIT
S-lock(B) → WAIT

After T1 commits:

T1 releases locks

T2:
S-lock(A)
S-lock(B)
Read balances
Unlock(A)
Unlock(B)

Why Strict 2PL is suitable

Strict 2PL keeps exclusive locks until commit. Therefore, T2 cannot read intermediate values while T1 is transferring money.

For example, T2 will not see:

A = ₹900
B = old balance

Instead, T2 sees either the balance before the transfer or the complete updated balance after T1 commits.

Conclusion

Strict 2PL ensures consistency, prevents dirty reads and dirty writes, and makes the banking transaction reliable.

Q10. Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.

Pseudocode

Algorithm Heuristic_Query_Optimization(Query)

Input:

SQL Query

Output:

Optimized Query Execution Plan

1. Parse the SQL query.
2. Create an initial query tree.
3. Identify all selection conditions.
4. For each selection condition:
 Push the selection as close as possible to the base relation.
5. Identify required attributes.
6. Push projection operations down to remove unnecessary attributes early.
7. Identify all join operations.
8. Estimate the size of intermediate relations.
9. Order joins:
 Perform joins involving smaller intermediate relations first.
10. Remove unnecessary operations.
11. Generate the optimized query tree.
12. Return the optimized execution plan.

Example

Initial operations:

Join
|
Selection
|
Projection
|
Relations

After heuristic optimization:

Projection
|
Join
/ \
smaller smaller
relation relation
↑ ↑
Selection Selection

Main heuristic rules

1. Push selection down:

Reduces the number of tuples processed early.

2. Project early:

Removes unnecessary attributes and reduces tuple size.

3. Order joins by smaller intermediate results:

Reduces the amount of intermediate data generated.

Conclusion

Heuristic optimization improves query performance by reducing intermediate results, disk I/O, memory usage, and processing time. The optimized plan is feasible because it applies the given constraints in a logical order.